

# Vấn đề 3: Phân bố bộ nhớ FLASH/RAM

Triển khai SOPWM trên TMS320F280049C

VietNH | epwm\_svm

June 28, 2026

## Abstract

Tài liệu riêng phân tích bài toán phân bố bộ nhớ khi triển khai SOPWM: vì sao đây là vấn đề cần giải quyết, cấu hình linker/CCS cụ thể trên F280049C, và đóng góp trực tiếp của mã nguồn (`sopwm.c/h`, `28004x_generic_flash_lnk.cmd`).

## Contents

|          |                                                                           |          |
|----------|---------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Ngữ cảnh ngắn</b>                                                      | <b>2</b> |
| <b>2</b> | <b>Vấn đề 3: Phân bố bộ nhớ FLASH/RAM</b>                                 | <b>2</b> |
| 2.1      | Phát biểu vấn đề                                                          | 2        |
| 2.2      | Cấu hình bộ nhớ trong dự án                                               | 3        |
| 2.2.1    | Bản đồ vật lý F280049C (trích <code>28004x_generic_flash_lnk.cmd</code> ) | 3        |
| 2.2.2    | Cấu hình section trong file liên kết                                      | 3        |
| 2.2.3    | Ràng buộc build CCS                                                       | 4        |
| 2.3      | Đóng góp của mã nguồn                                                     | 4        |
| 2.3.1    | Minh họa placement trong <code>sopwm.c</code>                             | 4        |
| 2.4      | Sơ đồ phân vùng logic                                                     | 4        |
| 2.5      | Chi tiết dung lượng LUT                                                   | 4        |
| 2.6      | Kết quả và hệ quả thiết kế                                                | 6        |

# 1 Ngữ cảnh ngắn

SOPWM cần hai lớp dữ liệu: (1) LUT góc tĩnh theo  $(N, m)$ ; (2) bảng lịch CMP động cho DMA. Hai lớp này có ràng buộc vùng nhớ khác nhau — phần dưới phân tích chi tiết.

$$Q1: \theta_i = \alpha_i; \quad Q2: 180^\circ - \alpha_{M+1-i}; \quad Q3: 180^\circ + \alpha_i; \quad Q4: 360^\circ - \alpha_{M+1-i}. \quad (1)$$

## 2 Vấn đề 3: Phân bố bộ nhớ FLASH/RAM

### 2.1 Phát biểu vấn đề

Triển khai SOPWM trên TMS320F280049C không chỉ cần “có đủ byte” — mỗi loại dữ liệu phải nằm đúng **loại bộ nhớ vật lý** và đúng **section liên kết**, nếu không hệ thống sẽ lỗi im lặng hoặc build thất bại. Đây là bài toán cấu trúc, không phải tối ưu phụ.

**(a) Khối lượng LUT góc lớn so với RAM LS.** Mỗi cặp  $(N, m)$  cần lưu trước  $M = (N - 1)/2$  góc chuyển mạch (độ). Với năm mức  $N$  và 100 bước  $m$ , tổng cộng:

$$S_{\text{LUT,naive}} = 100 \times 4 \times (3 + 4 + 5 + 6 + 7) = 10\,000 \text{ byte}. \quad (2)$$

Nếu đặt toàn bộ LUT vào RAM (như bảng sin/cos tham chiếu trong module SVPWM trước đó), section `.bss/.const` dễ **tràn RAMLS5** — lỗi linker đã ghi nhận trong `log.md`.

**(b) Bảng lịch runtime không được nằm FLASH.** Sau khi tra LUT, `SOPWM_BuildSchedule()` ghi liên tục vào bảng `sopwm_sched` — nguồn burst của DMA. Dữ liệu này:

- thay đổi mỗi chu kỳ cơ bản (500 Hz);
- phải nằm trong vùng RAM mà **DMA bus master** truy cập được (GS RAM trên F28004x);
- cần **hai bản sao** (double buffer) để CPU ghi buffer inactive trong khi DMA đọc buffer active.

Đặt nhầm vào RAM LS hoặc FLASH sẽ khiến DMA đọc sai, hoặc ghi CMP ra giá trị 0xFFFF.

**(c) Xung đột nhu cầu truy cập.** Cùng một lúc hệ thống cần:

| Thành phần                    | Truy cập           | Ràng buộc vùng            |
|-------------------------------|--------------------|---------------------------|
| LUT góc                       | CPU đọc, không đổi | FLASH <code>.const</code> |
| <code>sopwm_sched</code>      | CPU ghi + DMA đọc  | RAMGS, section riêng      |
| Stack / biến debug            | CPU R/W            | RAMLS / <code>.bss</code> |
| Mã <code>BuildSchedule</code> | CPU thực thi       | FLASH <code>.text</code>  |

(d) **Chi phí bộ nhớ tăng theo tham số thiết kế.** Nếu lưu thẳng 4M góc đã mở rộng (thay vì  $M$  góc Q1), dung lượng LUT nhân bốn:

$$S_{\text{LUT,full}} = 4 \times S_{\text{LUT,naive}} = 40 \text{ kB}. \quad (3)$$

Tương tự, bảng lịch tăng tuyến tính theo  $N_{\text{carr}}$  và số pha:

$$S_{\text{sched}} = N_{\phi} \times N_{\text{buf}} \times N_{\text{carr}} \times \text{sizeof}(\text{SOPWM\_CycleCmp\_t}). \quad (4)$$

Với  $N_{\phi} = 3$ ,  $N_{\text{buf}} = 2$ ,  $N_{\text{carr}} = 100$ , mỗi phần tử 4 byte:

$$S_{\text{sched}} = 3 \times 2 \times 100 \times 4 = 2400 \text{ byte}. \quad (5)$$

**Tóm lại**, vấn đề cần giải quyết là: *phân tách dữ liệu tĩnh (LUT) và dữ liệu động (bảng lịch DMA), map đúng section trong linker, đồng thời tối giảm footprint nhờ đối xứng quarter-wave — không tính góc runtime trên CPU.*

## 2.2 Cấu hình bộ nhớ trong dự án

### 2.2.1 Bản đồ vật lý F280049C (trích 28004x\_generic\_flash\_lnk.cmd)

Table 1: Vùng bộ nhớ on-chip liên quan SOPWM.

| Symbol linker    | Địa chỉ gốc | Dung lượng      | Vai trò trong SOPWM                                    |
|------------------|-------------|-----------------|--------------------------------------------------------|
| FLASH_BANK0_SEC4 | 0x084000    | 4 KiB           | Section <code>.const</code> — LUT                      |
| RAMGS0           | 0x00C000    | 8 KiB           | Section <code>ramgs0</code> — <code>sopwm_sched</code> |
| RAMLS5           | 0x00A800    | 2 KiB           | <code>.bss</code> , <code>.data</code> — biến toàn cục |
| RAMM1            | 0x000400    | $\approx 1$ KiB | Stack                                                  |

### 2.2.2 Cấu hình section trong file liên kết

File 28004x\_generic\_flash\_lnk.cmd khai báo hai section **do dự án thêm** (ngoài mặc định C2000Ware):

```
/* PAGE 1 -- MEMORY block */
RAMGS0 : origin = 0x00C000, length = 0x002000    /* 8 KiB */

/* SECTIONS */
.const   : > FLASH_BANK0_SEC4, PAGE = 0, ALIGN(4) /* const
float LUT */
ramgs0   : > RAMGS0, PAGE = 1 /* DMA
source table */
ramgs1   : > RAMGS1, PAGE = 1 /* du phong
mo rong */
```

**Lý do chọn FLASH\_BANK0\_SEC4 cho `.const`.** Sector 4 KiB đủ chứa  $\approx 10$  KiB LUT (chiếm  $\approx 24\%$  sector), tách biệt khỏi `.text` (SEC2/3/5) và `.cinit` (SEC1), tránh phải gom mã và hằng số vào cùng sector gây khó mở rộng.

**Lý do chọn RAMGS0 cho bảng lịch.** Theo TRM F28004x, DMA có thể truy cập Global Shared RAM (GS). `sopwm_sched` chiếm 2400 byte trên 8192 byte RAMGS0 ( $\approx 29\%$ ), còn dư cho biến DMA khác. Section `ramgs1` được khai báo sẵn trên RAMGS1 (8 KiB) cho mở rộng sau này.

### 2.2.3 Ràng buộc build CCS

- Cấu hình **CPU1\_FLASH** (hoặc tương đương) dùng `28004x_generic_flash_lnk.cmd`.
- `Flash_initModule(..., DEVICE_FLASH_WAITSTATES)` trong `device.c`: `WAITSTATES = 4 @ 100 MHz` — LUT đọc từ FLASH vẫn đủ nhanh vì `BuildSchedule` chỉ chạy 500 Hz, không phải 50 kHz.
- DMA `srcWrapSize = 200` trong `SysConfig` khớp  $2 \times N_{\text{carr}}$  word nguồn (double buffer liên tiếp trong RAM).

## 2.3 Đóng góp của mã nguồn

Bảng 2 tóm tắt cách code giải quyết vấn đề phân bố bộ nhớ.

### 2.3.1 Minh hoạ placement trong `sopwm.c`

LUT — compiler đặt vào `.const` (FLASH):

```
const float SOPWM_LUT_N7[SOPWM_M_COUNT][3] = {
    /* m=0.01 */ {60.118f, 80.177f, 80.364f},
    /* ... 100 rows ... */
};
```

Bảng lịch — placement explicit sang RAMGS:

```
#pragma DATA_SECTION(sopwm_sched, "ramgs0")
SOPWM_CycleCmp_t sopwm_sched[3][2][SOPWM_N_CARR];
```

`BuildSchedule` — đọc FLASH, ghi RAMGS (buffer inactive):

```
void SOPWM_BuildSchedule(float m, uint16_t N, uint16_t tbprd)
{
    float base_deg[7];
    float all_angles[28]; /* 4*7 max -- stack, không FLASH */
    n_base = SOPWM_GetAngles(m, N, base_deg); /* đọc LUT FLASH */
    /* ... mô hình quarter-wave vào all_angles[] ... */
    sopwm_bin_lut(all_angles, total, tbprd,
                  sopwm_sched[0][sopwm_sched_next]); /* ghi RAMGS */
}
```

## 2.4 Sơ đồ phân vùng logic

## 2.5 Chi tiết dung lượng LUT

Chỉ số tra cứu (code):

$$i_m = \text{round}(100m - 1), \quad i_m \in [0, 99]. \quad (6)$$

Table 2: Đóng góp mã nguồn cho bài toán bộ nhớ.

| Thành phần       | File                         | Đóng góp                                                                                                                                                                                                          |
|------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LUT tĩnh         | sopwm.c                      | Năm mảng <code>const float SOPWM_LUT_N*</code> ( $\approx 10$ kB) tự động vào <code>.const/FLASH</code> ; không dùng <code>malloc</code> , không copy sang RAM lúc khởi động.                                     |
| Tra cứu $O(1)$   | sopwm.h                      | <code>SOPWM_GetAngles()</code> <code>static inline</code> : map $m \rightarrow i_m = \text{round}(100m - 1)$ , <code>switch(N)</code> — thời gian xác định, không stack frame.                                    |
| Placement DMA    | sopwm.c                      | <code>#pragma DATA_SECTION(sopwm_sched, "ramgs0")</code> buộc linker đặt bảng vào RAMGS0; comment ghi rõ yêu cầu linker.                                                                                          |
| Kiểu dữ liệu gọn | sopwm.h                      | <code>SOPWM_CycleCmp_t = 2 × uint16_t</code> (4 byte/slot), không dùng <code>float</code> trong bảng DMA — giảm $S_{\text{sched}}$ và khớp burst DMA (2 word).                                                    |
| Double buffer    | sopwm.c/h                    | <code>sopwm_sched[3][2][SOPWM_N_CARR];</code> <code>sopwm_sched_active,</code> <code>sopwm_sched_next;</code> <code>BuildSchedule</code> ghi <code>[next]</code> , ISR hoán đổi — tránh ghi đè vùng DMA đang đọc. |
| Tiết kiệm FLASH  | sopwm.c                      | <code>SOPWM_BuildSchedule()</code> : chỉ đọc $M$ góc Q1 từ LUT, mở rộng $4M$ góc bằng (1) trên stack ( <code>all_angles[28]</code> ) — không lưu $4M$ góc/cấp $m$ trong FLASH.                                    |
| Linker           | 28004x_generic_flash_link.ld | Mã <code>cmd const</code> sang FLASH, <code>ramgs0</code> sang RAMGS0; tách LUT khỏi RAMLS tránh tràn như SVPWM.                                                                                                  |

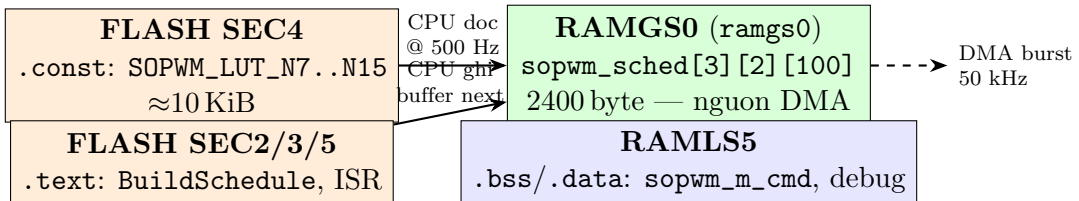


Figure 1: Phân vùng logic: LUT tĩnh trên FLASH; bảng lịch động trên RAMGS; CPU là cầu nối, DMA đọc trực tiếp RAMGS.

Table 3: Dung lượng từng bảng SOPWM\_LUT\_N\*.

| Bảng          | Kích thước     | Phần tử    | Bytes        |
|---------------|----------------|------------|--------------|
| SOPWM_LUT_N7  | $100 \times 3$ | 300        | 1200         |
| SOPWM_LUT_N9  | $100 \times 4$ | 400        | 1600         |
| SOPWM_LUT_N11 | $100 \times 5$ | 500        | 2000         |
| SOPWM_LUT_N13 | $100 \times 6$ | 600        | 2400         |
| SOPWM_LUT_N15 | $100 \times 7$ | 700        | 2800         |
| <b>Tổng</b>   |                | 2500 float | <b>10000</b> |

## 2.6 Kết quả và hệ quả thiết kế

1. **Linker thành công** với LUT trên FLASH và bảng lịch trên RAMGS — không tràn RAMLS như phiên bản SVPWM.
2. **DMA hoạt động ổn định** vì nguồn nằm đúng GS RAM; `srcWrapSize=200` khớp layout [3 pha][2 buf][100] khi mỗi kênh DMA phục vụ một pha.
3. **Footprint LUT giảm**  $\times 4$  nhờ lưu  $M$  góc Q1 + mở rộng runtime; footprint bảng lịch giảm nhờ `uint16_t` thay vì `float` cho CMP.
4. **Chi phí mở rộng có thể dự báo:** tăng  $N_{\text{carr}}$  từ 100 lên 200 sẽ đẩy  $S_{\text{sched}}$  lên 4800 byte; tăng số mức  $m$  từ 100 lên 1000 sẽ nhân mười dung lượng FLASH LUT.

*Kiểm tra sau build:* trong CCS, cửa sổ **Memory Browser** xác nhận địa chỉ `sopwm_sched` thuộc vùng `0x00C000–0x00DFFF`; map file liệt kê `SOPWM_LUT_N*` trong section `.const`.

## Tài liệu liên quan

- `pwm/sopwm/sopwm.c`, `sopwm.h`
- `28004x_generic_flash_lnk.cmd`
- `docs/bao_cao_sopwm_f280049c.tex` — báo cáo tổng hợp